UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Implementação de uma Interface Gráfica em Python/Tkinter para o Algoritmo de Estereotomografia

Gabriel Cruz de Lavor

Orientador: Prof. Dr. Tiago Tavares Leite Barros

Natal, RN, 10 de dezembro de 2025

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Implementação de uma Interface Gráfica em Python/Tkinter para o Algoritmo de Estereotomografia

Gabriel Cruz de Lavor

Orientador: Prof. Dr. Tiago Tavares Leite Barros

Trabalho de Conclusão de Curso de Graduação na modalidade Monografia, submetido como parte dos requisitos necessários para conclusão do curso de Engenharia de Computação pela Universidade Federal do Rio Grande do Norte (UFRN/CT).

Natal, RN, 10 de dezembro de 2025

Ficha Catalográfica

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Lavor, Gabriel Cruz de.
Implementação de uma interface gráfica em python/tkinter para o algoritmo de estereotomografia / Gabriel Cruz de Lavor. – 2025.
30 f.: il.

Monografia (graduação) – Universidade Federal do Rio Grande do Norte, Centro de Tecnologia, Curso de Engenharia de Computação, Natal, RN, 2025.
Orientação: Prof. Dr. Tiago Tavares Leite Barros.

1. Estereotomografia – Monografia. 2. Processamento Sísmico – Monografia. 3. Picking Automático – Monografia. 4. Interface Gráfica (GUI) – Monografia. 5. Imageamento Sísmico – Monografia.
I. Barros, Tiago Tavares Leite. II. Título.

RN/UF/BCZM CDU 550.34:004

Elaborado por Fernanda de Medeiros Ferreira Aquino - CRB-15/301

Agradecimentos

Agradeço primeiramente a Deus, pela força, sabedoria e perseverança concedidas ao longo de toda esta caminhada. Sem sua presença, nada disso teria sido possível.

Expresso minha profunda gratidão ao meu orientador, pela confiança, pelo apoio constante, pela paciência e pelas valiosas contribuições que foram essenciais para a realização deste trabalho. Sua dedicação e incentivo tiveram papel fundamental na minha formação acadêmica e profissional.

Agradeço também à minha companheira, por todo amor, compreensão e suporte emocional durante todas as etapas deste projeto. Sua presença constante foi fonte de motivação e tranquilidade nos momentos mais desafiadores.

Por fim, deixo meus sinceros agradecimentos aos meus familiares e amigos, que sempre me apoiaram, torceram por mim e estiveram ao meu lado ao longo de toda a minha trajetória. Cada palavra de incentivo teve grande importância na conclusão deste trabalho.

Resumo

Este Trabalho de Conclusão de Curso apresenta o desenvolvimento de uma interface gráfica (GUI) em Python, utilizando a biblioteca Tkinter, para auxiliar na experimentação e aplicação do algoritmo de estereotomografia. O campo de imageamento sísmico tem se apoiado há muito tempo em algoritmos sofisticados para reconstruir estruturas subsuperficiais, com a estereotomografia surgindo como uma técnica poderosa para a construção de modelos de velocidade e caracterização do subsolo. A implementação de interfaces interativas constitui um recurso importante para o manuseio desses dados, permitindo que pesquisadores e estudantes explorem parâmetros, visualizem resultados e acompanhem o processamento de forma mais intuitiva.

A interface desenvolvida integra módulos essenciais para o fluxo de trabalho da estereotomografia, incluindo carregamento de dados, visualização de gathers e execução dos estágios de inicialização, localização e inversão. Um dos elementos de destaque é a presença de uma aba dedicada ao Picking Automático, que melhora significativamente a eficiência na detecção de eventos sísmicos e na extração de parâmetros, etapa crítica no processo de estereotomografia.

O sistema foi projetado com ênfase em modularidade, usabilidade e organização visual, permitindo que o usuário configure parâmetros de modelagem, acompanhe o comportamento do algoritmo e visualize os resultados intermediários e finais. Os testes conduzidos demonstram que a GUI facilita o fluxo de trabalho e torna o processo de experimentação com estereotomografia mais acessível, representando uma contribuição relevante tanto para fins educacionais quanto para pesquisas aplicadas em processamento sísmico.

Palavras-chave: Estereotomografia, Processamento Sísmico, Picking Automático, Interface Gráfica (GUI), Imageamento Sísmico.

Abstract

This Undergraduate Thesis presents the development of a graphical user interface (GUI) in Python, using the Tkinter library, to assist in the experimentation and application of the stereotomography algorithm. The field of seismic imaging has long relied on sophisticated algorithms to reconstruct subsurface structures, with stereotomography emerging as a powerful technique for building velocity models and characterizing the subsurface. The implementation of interactive interfaces constitutes an important resource for handling these data, allowing researchers and students to explore parameters, visualize results, and follow the processing in a more intuitive way.

The developed interface integrates essential modules for the stereotomography workflow, including data loading, gather visualization, and execution of the initialization, location, and inversion stages. One of the standout elements is the presence of a tab dedicated to Automatic Picking, which significantly improves the efficiency in detecting seismic events and extracting parameters, a critical step in the stereotomography process.

The system was designed with an emphasis on modularity, usability, and visual organization, allowing the user to configure modeling parameters, monitor the algorithm's behavior, and visualize intermediate and final results. The tests conducted demonstrate that the GUI facilitates the workflow and makes the experimentation process with stereotomography more accessible, representing a relevant contribution both for educational purposes and for applied research in seismic processing.

Keywords: Stereotomography, Seismic Processing, Automatic Picking, Graphical User Interface (GUI), Seismic Imaging.

Sumário

Ficha Catalográfica **3**

Lista de Figuras **iii**

1 Introdução **1**

2 Fundamentação Teórica **3**
2.1 Processamento Sísmico e Construção de Modelos de Velocidade 3
2.2 O Método da Estereotomografia 3
2.2.1 Espaço dos Dados e Espaço do Modelo 4
2.2.2 O Problema Inverso e Otimização 5
2.3 Picking de Eventos Sísmicos 5
2.4 Computação Aplicada à Geofísica 6
2.5 Interfaces Gráficas para Software Científico 7
2.6 Integração entre GUI, Picking e Estereotomografia 7

3 Trabalhos relacionados **9**
3.1 Trabalhos Clássicos em Estereotomografia 9
3.2 Softwares e Implementações Computacionais 9
3.3 Picking Automático e Ferramentas de Interpretação 11
3.4 Interfaces Gráficas para Aplicações Científicas 11
3.5 Posicionamento do Trabalho Proposto 12

4 Problema **13**
4.1 Interface Gráfica dos softwares científicos de alto desempenho 13
4.2 Pré-processamento dos dados 13
4.3 Fragmentação das ferramentas 14

5 Implementação **15**
5.1 Arquitetura do Sistema 15
5.2 Tecnologias e Bibliotecas Utilizadas 16
5.3 Gerenciamento de Concorrência (Threading) 16
5.4 Módulo de Picking Automático 17
5.5 Módulo de Estereotomografia 18
5.6 Visualização de Resultados e Controle de Qualidade 19

6 Experimentos e Resultados **21**
6.1 Metodologia de Validação 21
6.2 Validação do Módulo de Picking Automático 21
6.3 Execução da Inversão e Inspeção da Convergência 22
6.4 Análise de Usabilidade e Desempenho 24

7 Conclusão **27**

Referências bibliográficas **29**

Lista de Figuras

2.1 Representação esquemática do princípio da Estereotomografia. 4

5.1 Arquitetura do sistema 16
5.2 Tela da interface gráfica mostrando o Picking Automático. 17
5.3 Tela da interface gráfica mostrando a Estereotomografia. 18

6.1 Modelo de Velocidade gerado pelo Picking Automático. 22
6.2 Modelo da Função Custo gerado pela inversão. 23
6.3 Modelo de Velocidade Final gerado pela inversão. 23
6.4 Modelo de Velocidade Equivalente gerado pela inversão. 24

Capítulo 1

Introdução

O processamento sísmico desempenha um papel fundamental na caracterização da subsuperfície, sobretudo em aplicações relacionadas à exploração de petróleo, gás e estudos geofísicos avançados. Entre os métodos cinemáticos utilizados para estimar modelos de velocidade, destaca-se a estereotomografia, um processo inverso que combina simultaneamente tempos de trânsito e inclinações locais (slopes) extraídos de eventos de reflexão sísmica, permitindo reconstruir modelos macro de velocidade robustos e estáveis (Billette & Lambaré 1998). Ao incorporar as inclinações locais (slopes) além dos tempos de chegada, o método aumenta a sensibilidade à geometria dos refletores e reduz ambiguidades, tornando-se especialmente adequado para regiões de forte heterogeneidade (Chauris & Noble 2001).

Apesar de sua eficácia, o desempenho da estereotomografia depende criticamente da qualidade da preparação dos dados, em particular da etapa de picking dos eventos sísmicos. O picking manual continua sendo um gargalo operacional, pois é um processo exaustivo, sujeito à variabilidade interpretativa e inadequado para grandes volumes de dados. Como alternativa, algoritmos automáticos vêm sendo amplamente estudados, explorando atributos como coerência, energia e derivações locais do traço sísmico para identificar tempos e slopes de forma mais objetiva e repetível (Yilmaz 2001). A integração de ferramentas automáticas de picking ao fluxo de trabalho tem se mostrado essencial para acelerar a etapa de pré-processamento e aumentar a precisão dos parâmetros utilizados na inversão.

Com o avanço de tecnologias de desenvolvimento de software científico, cresce também a demanda por interfaces gráficas capazes de tornar esses fluxos de trabalho mais acessíveis e eficientes (Siahaan et al. 2025). Nesse cenário, Python destaca-se como linguagem altamente produtiva, com amplo ecossistema científico e suporte multiplataforma. Entre suas bibliotecas, o Tkinter fornece uma base sólida para o desenvolvimento de Interfaces Gráficas de Usuário (GUI), permitindo implementar aplicações interativas de baixo custo computacional e fácil manutenção (Morse & Middleton 1994).

Além das iniciativas acadêmicas que avançam nesse campo, como a Seismic Waveform Inversion Toolbox (SWIT), que combina rotinas de inversão em Fortran com uma GUI em Python, há também soluções comerciais amplamente reconhecidas, como os softwares da NORSAR (e.g., NORSAR-3D) e da DUG (e.g., DUG Insight). Esses ambientes oferecem interfaces gráficas sofisticadas para modelagem, processamento e inter-

pretação sísmica. No entanto, apesar de seu alto nível de maturidade, tais ferramentas são geralmente proprietárias, possuem custos elevados e não fornecem acesso direto às rotinas internas, o que limita sua utilização em ambientes acadêmicos e de pesquisa aberta. Assim, há um espaço importante para o desenvolvimento de soluções experimentais, didáticas e flexíveis que complementem essas plataformas, especialmente no contexto de métodos específicos como a estereotomografia.

Diante desse panorama, o presente Trabalho de Conclusão de Curso tem como objetivo implementar uma interface gráfica em Python/Tkinter destinada a auxiliar a execução do algoritmo de estereotomografia, reunindo em um único ambiente módulos para carregamento de dados sísmicos, inspeção visual de gathers, configuração de parâmetros e acompanhamento das etapas do processo inverso. Um dos elementos centrais da interface desenvolvida é a inclusão de uma aba dedicada ao picking automático, capaz de extrair parâmetros cinemáticos fundamentais de forma rápida e eficiente. Dessa maneira, a ferramenta proposta integra, em um fluxo contínuo, funcionalidades que tipicamente se encontram dispersas em múltiplos softwares ou demandam conhecimento avançado de linha de comando (Bian et al. 2022).

O desenvolvimento desta GUI contribui tanto para o ensino dos fundamentos da estereotomografia quanto para sua experimentação prática, oferecendo um ambiente modular e de fácil manipulação. Além disso, demonstra o potencial de Python/Tkinter como plataforma de prototipação de ferramentas didáticas e científicas, abrindo caminho para futuras extensões, como a incorporação do método CRS diretamente na interface, permitindo que todo o processo de tomografia seja realizado a partir de um único dado sísmico, e a adaptação da aplicação para execução em supercomputadores, incluindo suporte a servidores remotos, e a integração com sistemas de filas, como sbatch.

Capítulo 2

Fundamentação Teórica

A construção de ferramentas computacionais voltadas para a geofísica exige uma compreensão abrangente de diversos domínios científicos interconectados. Dessa forma, este capítulo aborda os conceitos centrais da estereotomografia e do processamento de dados sísmicos, bem como a importância de interfaces gráficas de usuário (GUIs) para otimizar fluxos de trabalho complexos.

2.1 Processamento Sísmico e Construção de Modelos de Velocidade

O processamento sísmico tem como objetivo transformar dados brutos adquiridos em campo em informações úteis para a interpretação geológica. Uma das etapas mais importantes desse fluxo é a construção de modelos de velocidade, que descrevem como as ondas sísmicas se propagam pela subsuperfície. Esses modelos são fundamentais para procedimentos como migração, análise de velocidade e inversão sísmica (Yilmaz 2001). A qualidade do modelo de velocidade influencia diretamente a capacidade de localizar interfaces geológicas e identificar possíveis reservatórios.

Modelos de velocidade podem ser obtidos tanto pela análise de empilhamento quanto por métodos de inversão (Billette & Lambaré 1998). Dentre estes últimos, destacam-se técnicas cinemáticas, como a tomografia de tempos de trânsito (travel-time tomography), e métodos mais sofisticados que também utilizam inclinações locais das ondas, como é o caso da estereotomografia (Chauris & Noble 2001).

2.2 O Método da Estereotomografia

A tomografia sísmica é uma técnica de inversão utilizada para estimar as propriedades físicas do subsolo, predominantemente a velocidade de propagação das ondas, a partir de dados registrados em diversas geometrias de aquisição, incluindo arranjos de superfície, medições em poços e redes de fundo oceânico (como OBNs e OBCs). Enquanto a tomografia convencional de tempos de trânsito se baseia na minimização dos resíduos de tempo entre fonte e receptor, a estereotomografia, proposta por Billette & Lambaré

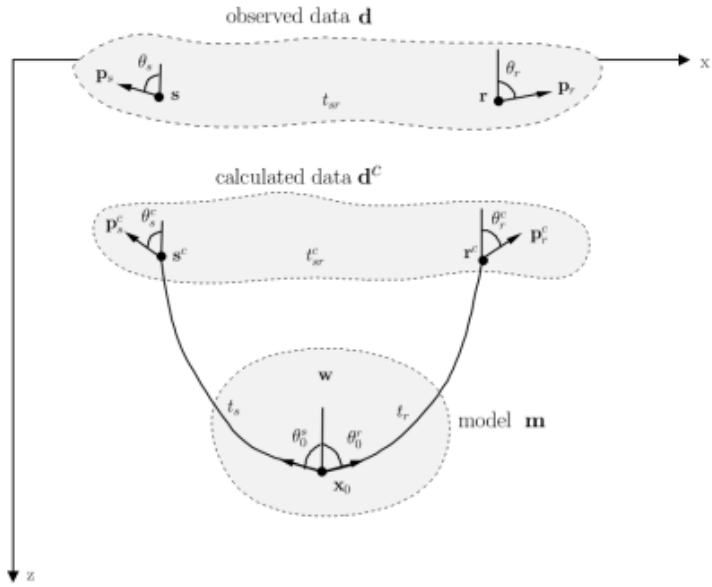
(1998), introduz uma abordagem mais rigorosa ao incorporar informações cinemáticas de ordem superior: as inclinações locais (*slopes*) dos eventos sísmicos.

A principal premissa do método é que o modelo de velocidade não precisa ser restringido por interfaces geológicas contínuas interpretadas. Em vez disso, o método baseia-se em "eventos localmente coerentes", permitindo uma inversão mais estável e de maior resolução em meios geologicamente complexos (Chauris & Noble 2001).

2.2.1 Espaço dos Dados e Espaço do Modelo

Para compreender a formulação matemática da estereotomografia, é fundamental distinguir o vetor de observações (dados) do vetor de parâmetros a serem invertidos (modelo). A Figura 2.1 ilustra a relação geométrica entre esses dois domínios. (a) Na superfície, os parâmetros observados: posições (s,r) e inclinações (p_s,p_r). (b) Em profundidade, o modelo representado pelo ponto de difração X e os segmentos de raios.

Figura 2.1: Representação esquemática do princípio da Estereotomografia.



Fonte: Adaptado de Barros (2014).

O Espaço dos Dados (d): Diferente da tomografia clássica que utiliza triplas (s,r,t), a estereotomografia define um dado como uma quíntupla de parâmetros cinemáticos extraídos dos sismogramas. Um evento i é definido por:

$$d_i = (s,r,t,p_s,p_r)_i \tag{2.1}$$

Onde:

- s,r: Coordenadas da fonte e do receptor na superfície;
- t: Tempo de trânsito duplo observado (Two-way traveltime);

- p_s: Inclinação do evento no domínio do tiro comum ($\frac{\partial t}{\partial s}$);
- p_r: Inclinação do evento no domínio do receptor comum ($\frac{\partial t}{\partial r}$).

As inclinações p_s e p_r são fundamentais, pois restringem o ângulo de incidência dos raios na superfície, reduzindo drasticamente a ambiguidade do problema inverso.

O Espaço do Modelo (m): O modelo na estereotomografia é híbrido, composto pelos parâmetros do meio físico e pela geometria dos raios. O vetor modelo m é definido por:

$$m = (m_{vel}, m_{ray}) \tag{2.2}$$

- m_{vel}: Representa o campo de velocidades $C(x,z)$, geralmente parametrizado por funções de base suaves, como B-splines cardinais, para garantir a diferenciabilidade necessária ao traçamento de raios.
- m_{ray}: Para cada dado observado, o modelo propõe um par de segmentos de raios que se originam em um ponto de difração hipotético $X(x,z)$ em profundidade e emergem na superfície. Este par é descrito pela posição de X, pelos ângulos de saída e pelos tempos de trânsito unidirecionais.

2.2.2 O Problema Inverso e Otimização

O objetivo da inversão é encontrar o modelo m que minimize a discrepância entre os dados observados (d_{obs}) e os dados calculados (d_{calc}) pelo traçamento de raios no modelo proposto. A função custo $J(m)$ é tipicamente definida pela norma L_1 dos resíduos, acrescida de termos de regularização:

$$J(m) = \frac{1}{2}\|d_{calc}(m) - d_{obs}\|^2 + \lambda_1 R_1(m) \tag{2.3}$$

A minimização desta função não linear requer o cálculo do gradiente de J em relação aos parâmetros do modelo. Abordagens modernas, como descrito por Chauris & Noble (2001), utilizam o **Método do Estado Adjunto** (*Adjoint-State Method*) para calcular esse gradiente de forma eficiente, evitando o custo proibitivo do cálculo explícito da matriz Jacobiana em problemas de grande escala.

2.3 Picking de Eventos Sísmicos

O picking sísmico consiste em identificar, dentro de um registro sísmico, eventos relevantes, como tempos de chegada e máximos de coerência. Tradicionalmente realizado de maneira manual, esse processo é repetitivo, lento e suscetível a variações entre intérpretes. O avanço dos métodos automáticos visa minimizar tais limitações, empregando métricas como energia, similaridade, coerência e análise de derivadas locais (Zamorouev 1999). No contexto da estereotomografia, a correta identificação desses eventos é particularmente importante, pois influencia diretamente a qualidade dos parâmetros cinemáticos usados na inversão (Chauris & Noble 2001).

A abordagem adotada neste trabalho baseia-se em um dos princípios mais eficazes no picking automático moderno: a análise de painéis de coerência (semblance). Esse método

estima a similaridade entre traços vizinhos dentro de uma janela temporal móvel, avaliando diferentes inclinações possíveis. Em termos computacionais, essa operação equivale a aplicar, para cada tempo e posição, um empilhamento orientado segundo um conjunto de slopes de teste, verificando qual direção maximiza a coerência. Técnicas como Slant Stack ou análises baseadas em atributos estruturais são frequentemente utilizadas para esse cálculo, uma vez que permitem explorar conjuntamente energia e alinhamento das formas de onda (Yilmaz 2001). Assim, regiões onde o valor de semblance ultrapassa um limiar são interpretadas como eventos sísmicos coerentes e, portanto, candidatos válidos ao processo de picking.

Complementarmente, outros métodos extraem estimativas diretas da inclinação local usando derivadas espaciais do campo de onda, aproveitando o fato de que a inclinação representa a variação lateral do tempo de trânsito. Abordagens dessa natureza oferecem resultados mais estáveis quando a razão sinal-ruído é favorável, permitindo uma caracterização cinemática mais detalhada sem recorrer a janelas temporais extensas (Bian et al. 2022).

No pseudocódigo implementado neste trabalho — e que fundamenta o módulo de picking automático da interface — a lógica segue quatro etapas principais: (i) leitura do gather e normalização do traço; (ii) varredura temporal utilizando janelas móveis; (iii) cálculo da coerência ou amplitude média ponderada dentro da janela, incorporando a possibilidade de diferentes inclinações de teste; e (iv) seleção dos máximos locais que excedem o limiar configurado pelo usuário. Esse processo reproduz, de forma simplificada, o mecanismo de avaliação de energia descrito nos métodos clássicos de semblance, mas com uma execução otimizada para uso interativo dentro da GUI. A escolha dessa abordagem se justifica por sua robustez e ampla aplicabilidade em dados de reflexão, além de se alinhar às práticas discutidas nas referências contemporâneas de picking e interpretação automática.

Na interface gráfica desenvolvida, toda essa complexidade matemática é encapsulada em uma camada de abstração que permite ao usuário ajustar apenas os parâmetros mais importantes — como a largura da janela temporal e o limiar mínimo de coerência ou amplitude. Dessa forma, o usuário pode explorar rapidamente diferentes configurações, visualizar os eventos sobrepostos aos traços e validar imediatamente a consistência dos resultados antes de exportá-los para o módulo de estereotomografia. Esse fluxo reduz drasticamente o tempo de preparação dos dados, aumenta a repetibilidade do processo e minimiza erros nos vetores de entrada utilizados pela inversão.

2.4 Computação Aplicada à Geofísica

A implementação de métodos de inversão como a estereotomografia frequentemente requer algoritmos computacionalmente intensivos, em especial durante o traçamento de raios e na otimização do modelo. Trabalhos recentes destacam a importância de estratégias eficientes de programação, paralelismo e gerenciamento de memória para viabilizar aplicações práticas em cenários de grande escala (Barros et al. 2022). Além disso, a complexidade dos cálculos associados à estimativa de macro modelos de velocidade e à manipulação de grandes volumes de dados sísmicos reforça a necessidade de ferramentas

computacionais robustas e otimizadas, capazes de reduzir o tempo de processamento sem comprometer a acurácia dos resultados.

Mesmo quando a interface gráfica não executa diretamente o núcleo computacional, ela desempenha um papel fundamental como camada intermediária que organiza, configura e aciona processos externos. Nesse contexto, a GUI atua como um facilitador do fluxo de trabalho, permitindo que o usuário configure parâmetros, visualize resultados intermediários e acompanhe o progresso de algoritmos complexos de maneira intuitiva. A presença de elementos interativos, como abas dedicadas a Picking Automático ou módulos de visualização, não apenas melhora a experiência do usuário, mas também contribui para a confiabilidade e reprodutibilidade dos experimentos.

2.5 Interfaces Gráficas para Software Científico

O desenvolvimento de Interfaces Gráficas de Usuário (GUI) busca tornar mais acessíveis ferramentas que, de outra forma, requereriam uso direto de programação ou manipulação de arquivos de configuração. Em aplicações científicas, GUIs desempenham papel fundamental ao permitir que pesquisadores e estudantes explorem dados e configurem experimentos sem necessidade de conhecimento profundo de computação (Morse & Middleton 1994).

O Tkinter, biblioteca padrão do Python para construção de GUIs, destaca-se por sua simplicidade, portabilidade e integração nativa com o ecossistema da linguagem. Ele permite a criação de interfaces estruturadas, com menus, abas, botões, campos de entrada e elementos de visualização, sem a necessidade de frameworks externos. Trabalhos recentes demonstram que Python, aliado ao Tkinter, tem sido amplamente utilizado no desenvolvimento rápido de protótipos para aplicações científicas, graças à sua versatilidade e ampla comunidade de suporte (Siahaan et al. 2025).

Ao implementar abas dedicadas para diferentes funcionalidades, como a visualização de dados e execução do algoritmo e Picking Automático, a GUI se torna um ambiente completo para experimentação em Estereotomografia, integrando etapas essenciais do fluxo de trabalho de maneira organizada e didática.

2.6 Integração entre GUI, Picking e Estereotomografia

A integração entre interface gráfica, picking automático e o algoritmo de Estereotomografia contribui para uma abordagem moderna de análise sísmica. Essa combinação permite:

- facilitar o carregamento e inspeção visual dos dados;
- automatizar a seleção dos parâmetros utilizados na inversão;
- acompanhar a evolução do modelo ao longo das iterações;
- reduzir a complexidade operacional da técnica;
- tornar o processo acessível a usuários não especialistas em programação.

Ao disponibilizar tais ferramentas em um único ambiente, o sistema favorece tanto o aprendizado acadêmico quanto aplicações reais em pesquisa, ampliando o alcance e a aplicabilidade da Estereotomografia.

Capítulo 3

Trabalhos relacionados

O desenvolvimento de ferramentas computacionais que auxiliem no processamento sísmico tem sido objeto de diversos estudos nas últimas décadas, especialmente diante do crescimento do volume de dados e da complexidade geológica observada em ambientes de exploração. A estereotomografia, por sua capacidade de integrar informações cinemáticas detalhadas para a construção de macro modelos de velocidade, motivou diversas pesquisas tanto na sua execução quanto em sua implementação computacional. Este capítulo revisa a literatura pertinente para contextualizar o projeto dentro do cenário científico e tecnológico atual.

3.1 Trabalhos Clássicos em Estereotomografia

Os primeiros avanços consolidados na área foram apresentados por Billette e Lambaré (1998), que estabeleceram os fundamentos da Estereotomografia como um método robusto para a estimativa de macro-modelos de velocidade. O estudo demonstrou que a combinação de tempos e inclinações locais dos eventos sísmicos produz resultados superiores quando comparado a métodos tradicionais de tomografia de tempo de trânsito. Posteriormente, Chauris e Noble (2001) expandiram esse trabalho aplicando a técnica em dados bidimensionais mais complexos, explorando métodos de localização e refinamento dos eventos utilizando atributos cinemáticos extraídos diretamente dos dados.

Essas pesquisas estabeleceram a base teórica que sustenta a maioria dos estudos subsequentes sobre estereotomografia e inspiraram trabalhos focados em implementações computacionais mais eficientes, técnicas de otimização e métodos de traçamento de raios adequados a problemas de larga escala.

3.2 Softwares e Implementações Computacionais

Historicamente, o desenvolvimento de software para processamento sísmico acadêmico priorizou a eficiência computacional em detrimento da usabilidade. Um exemplo clássico e amplamente citado é o Seismic Unix (SU), mantido pelo Center for Wave Phenomena (CWP). Conforme documentado por Cohen e Stockwell (1999), o SU consiste em uma coleção robusta de ferramentas de linha de comando. Embora extremamente poderoso e gratuito, apresenta uma curva de aprendizado íngreme, exigindo profundo

conhecimento de shell scripting e manipulação de arquivos binários, sem fornecer uma interface gráfica nativa para gerenciamento de fluxos de trabalho.

Com a evolução das demandas da indústria e da pesquisa, começaram a surgir iniciativas que buscavam equilibrar desempenho computacional com acessibilidade. Nesse contexto, Fomel (2013) introduziu o pacote Madagascar, com foco em "pesquisa reprodutível", utilizando scripts SCons (baseados em Python) para a organização de fluxos de processamento. Apesar de representar um avanço em relação à integração com linguagens modernas, o Madagascar ainda depende fortemente de scripts e comandos de terminal, mantendo-se distante da experiência de uso oferecida por interfaces gráficas completas.

Uma alternativa mais recente e promissora é a Seismic Waveform Inversion Toolbox (SWIT), desenvolvida por Haipeng Li na USTC e posteriormente na Stanford University. Diferentemente de ferramentas acadêmicas tradicionais, o SWIT combina rotinas avançadas de modelagem e inversão em Fortran com uma camada de controle e visualização em Python, disponibilizando ainda uma GUI dedicada. Essa integração torna o SWIT acessível a usuários sem familiaridade com terminal, representando um movimento importante em direção à usabilidade no processamento sísmico moderno.

Paralelamente, softwares comerciais já consolidaram há décadas o uso de interfaces gráficas robustas, exemplificados pelas ferramentas da NORSAR e da DUG. Esses ambientes oferecem módulos integrados para modelagem sísmica, migração, FWI, análise cinemática e visualização interativa em múltiplas dimensões, demonstrando o impacto positivo que interfaces visuais têm na produtividade e na qualidade interpretativa. Entretanto, tais soluções, apesar de extremamente completas, possuem custos elevados e são voltadas predominantemente ao mercado industrial, dificultando seu acesso em ambientes acadêmicos.

Nesse panorama intermediário entre ferramentas altamente técnicas e soluções comerciais sofisticadas, destaca-se também o BotoSeis, desenvolvido na Universidade Federal do Pará (UFPA) por Lima, Garabito e Pinto. O BotoSeis funciona como uma camada gráfica sobre o SU, encapsulando rotinas de filtragem, ganho, normalização, visualização de gathers e montagem de pipelines dentro de uma interface amigável. Essa aproximação democratiza o uso do SU, antes limitado à linha de comando, promovendo um ambiente mais próximo dos padrões atuais de usabilidade e ensino.

Embora todos esses softwares representem avanços significativos, grande parte das ferramentas acadêmicas ainda opera de forma fragmentada ou depende de scripts extensos, exigindo experiência prévia em programação. Além disso, poucos ambientes integrados permitem manipular dados de estereotomografia de forma acessível, especialmente no que concerne ao pré-processamento, configuração de parâmetros e execução controlada das etapas de picking e inversão. Nesse contexto, torna-se evidente a necessidade de ferramentas mais didáticas e integradas para visualização, pré-processamento e execução de algoritmos de estereotomografia, especialmente em ambientes educacionais e de prototipação científica.

3.3 Picking Automático e Ferramentas de Interpretação

O picking de eventos sísmicos é um componente crítico para métodos cinemáticos como a Estereotomografia, pois fornece tempos e inclinações essenciais para a formulação do problema inverso. A literatura apresenta diversos métodos voltados a automatizar essa tarefa. Zamorouev (1999), por exemplo, explorou técnicas generalizadas para detecção automática de eventos utilizando atributos como coerência e derivadas espaciais. Yilmaz (2001), também descreve abordagens manuais e automáticas de picking, incluindo análises de semblance, slant stack e inspeção de gathers multicomponentes.

A integração de algoritmos robustos de picking automático à estereotomografi está alinhada com tendências modernas no desenvolvimento de software científico, onde GUIs frequentemente atuam como centros de controle para pipelines analíticos complexos, especialmente aqueles que incorporam aprendizado de máquina (Huang et al. 2022). Por exemplo, o software PyRefra disponibiliza funcionalidades de picking manual e automático para dados sísmicos de refração, além de ferramentas de exibição e processamento dentro de sua GUI (Zeyen & Léger 2024). Essa abordagem permite que geofísicos processem rapidamente grandes conjuntos de dados, gerem macromodelos de velocidade precisos e reduzam a variabilidade subjetiva associada às interpretações manuais.

3.4 Interfaces Gráficas para Aplicações Científicas

Nos últimos anos, Python tornou-se uma linguagem amplamente adotada no desenvolvimento de ferramentas científicas, graças à sua simplicidade, vasta biblioteca padrão e grande comunidade de usuários. Entre suas bibliotecas nativas, o Tkinter tem se destacado na criação de interfaces educacionais, protótipos de sistemas científicos e até aplicações distribuídas (Richards 2023). Embora existam soluções mais modernas para GUIs, como PyQt ou frameworks baseados em web, o Tkinter se diferencia por sua leveza, dependências mínimas e facilidade de integração com scripts existentes.

Nesse contexto, Grayson (2000) apresenta exemplos práticos do uso do Tkinter para construir interfaces robustas e interativas, enquanto Van Rossum e Drake (1995) ressaltam a flexibilidade do Python como linguagem para aplicações científicas. Seguindo esse caminho, o projeto ObsPy, descrito por Krischer (2015), tornou-se uma referência global em sismologia em Python. Apesar de focado em análise de terremotos, o sucesso do ObsPy demonstra que a linguagem é capaz de manipular grandes volumes de dados temporais e fornecer visualizações complexas. Entretanto, o ObsPy funciona predominantemente como uma biblioteca de scripts, sem oferecer uma interface independente com menus e botões (GUI).

O uso do Tkinter como wrapper (envelopes) para códigos científicos é defendido por Lutz (2010) como uma solução eficiente para garantir portabilidade e simplicidade de instalação em diferentes sistemas operacionais. Diferente de frameworks mais pesados, como Qt, o Tkinter permite que pesquisadores desenvolvam ferramentas leves para controlar códigos mais complexos em Fortran ou C — arquitetura esta adotada neste trabalho para gerenciar o algoritmo de estereotomografia.

3.5 Posicionamento do Trabalho Proposto

Os trabalhos relacionados evidenciam uma lacuna significativa entre algoritmos avançados de estereotomografia e ferramentas acessíveis para sua aplicação prática. O presente Trabalho de Conclusão de Curso busca preencher parte dessa lacuna ao desenvolver uma interface gráfica em Python/Tkinter, capaz de facilitar a interação com o algoritmo, em consonância com as tendências de automação e usabilidade discutidas na literatura.

A GUI proposta integra, em um único ambiente, a visualização de gathers sísmicos, a seleção e configuração de parâmetros do algoritmo, a execução do fluxo completo da estereotomografia e, de forma destacada, o Picking Automático. Ao reunir essas funcionalidades de maneira modular e intuitiva, a interface atua como um elo entre implementações computacionalmente complexas e o usuário final, tornando o processo mais didático, eficiente e acessível.

Capítulo 4

Problema

A análise e interpretação de dados sísmicos é um processo complexo que envolve diversas etapas e entre os métodos utilizados, a estereotomografia se destaca como uma ferramenta poderosa capaz de superar limitações de métodos convencionais (Billette & Lambaré 1998). No entanto, a sua sofisticação matemática e computacional impõe barreiras práticas significativas que dificultam sua adoção rotineira e o ensino acadêmico.

4.1 Interface Gráfica dos softwares científicos de alto desempenho

Para garantir a eficiência necessária no processamento de grandes volumes de dados, os algoritmos de estereotomografia são predominantemente implementados em linguagens compiladas de baixo nível (como Fortran ou C) e operados via interface de linha de comando (CLI). No contexto das melhores práticas para computação científica, o uso exclusivo de scripts e arquivos de configuração textual aumenta a probabilidade de erros de entrada (input errors) e dificulta a visualização imediata do impacto da alteração de parâmetros (Wilson et al. 2014). O geocientista acaba dedicando tempo demais a gerenciar a execução do código e a formatar arquivos, em vez de concentrar-se na análise geológica dos resultados.

4.2 Pré-processamento dos dados

Talvez o problema mais crítico do ponto de vista operacional, é a etapa de pré-processamento dos dados, especificamente o picking (seleção) de eventos sísmicos. A estereotomografia exige não apenas o tempo de trânsito (t), mas também a inclinação local (p) das frentes de onda (Billette & Lambaré 1998). Realizar a identificação e a seleção desses pares de dados (t, p) manualmente em milhares de traços sísmicos é uma tarefa exaustiva, lenta e extremamente sujeita à subjetividade do intérprete. A falta de ferramentas que automatizem essa extração de atributos cria um gargalo no fluxo de trabalho, onde o tempo de preparação dos dados supera largamente o tempo de computação da inversão (Sabbione & Velis 2010).

4.3 Fragmentação das ferramentas

Frequentemente, o pesquisador precisa utilizar um software para visualizar o dado (ex: Seismic Unix), outro script para tentar realizar o picking, um editor de texto para configurar os parâmetros da inversão e, finalmente, o binário da estereotomografia para processar. Essa descontinuidade no fluxo de trabalho impede uma análise iterativa e ágil. Portanto, o problema central que este trabalho busca resolver é a ausência de um ambiente integrado e visual que permita ao usuário realizar o ciclo completo da estereotomografia, desde a seleção automática de dados (picking) até a execução e controle de qualidade da inversão, sem a necessidade de manipular complexos scripts ou arquivos de texto manualmente.

Capítulo 5

Implementação

A implementação da interface gráfica proposta neste trabalho foi desenvolvida em Python utilizando a biblioteca Tkinter, com o objetivo de fornecer um ambiente integrado e acessível para a execução e experimentação do algoritmo de Estereotomografia. A construção da GUI foi planejada de forma modular, permitindo que diferentes funcionalidades, como visualização de dados, execução da inversão, carregamento de arquivos e picking automático, fossem organizadas em abas independentes dentro de um único sistema.

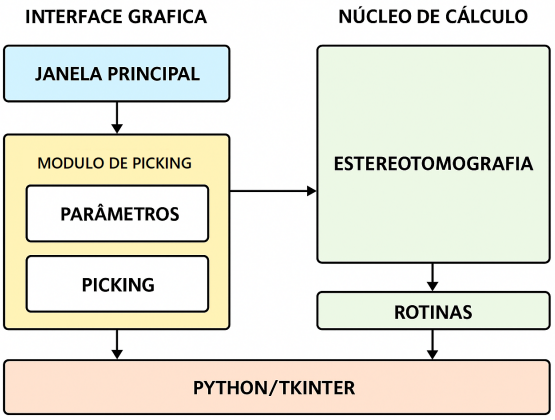
A solução proposta busca preencher uma lacuna existente entre softwares de alto desempenho, voltados a especialistas, e interfaces acessíveis que possam ser utilizadas em ambientes acadêmicos e de pesquisa aplicada. Ao oferecer uma GUI intuitiva, com funcionalidades específicas para picking automático, pretende-se contribuir para a melhoria do fluxo de trabalho em estereotomografia, ampliando sua aplicabilidade e reduzindo as barreiras de entrada para novos usuários.

5.1 Arquitetura do Sistema

O sistema foi projetado seguindo uma arquitetura modular que separa a camada de apresentação (Interface Gráfica) da camada de processamento intensivo (Núcleo de Cálculo). A aplicação atua como um "envelope"(wrapper) de alto nível, gerenciando a execução de rotinas complexas e a visualização de dados.

A estrutura do código, contida no script principal, utiliza o padrão de widgets hierárquicos. A janela principal (root) atua como o contêiner mestre, gerenciando o ciclo de vida da aplicação. Para organizar os diferentes fluxos de trabalho — configuração, execução e pré-processamento — optou-se pelo uso de um sistema de abas (Notebook), implementado através do módulo ttk (Themed Tkinter). Isso permite que o usuário alterne entre a parametrização da inversão e a realização do picking de forma intuitiva, mantendo o contexto da aplicação.

Figura 5.1: Arquitetura do sistema



Fonte: Elaborado pelo autor (2025)

5.2 Tecnologias e Bibliotecas Utilizadas

A implementação foi realizada integralmente em Python 3, escolhida por sua robustez e vasta disponibilidade de bibliotecas científicas. As principais dependências do projeto e suas respectivas funções são:

- **Tkinter e Ttk:** Bibliotecas nativas da linguagem utilizadas para a construção de todos os elementos visuais (botões, menus, caixas de diálogo). A escolha por bibliotecas nativas garante que o software execute em qualquer sistema operacional (Windows, Linux, macOS) sem a necessidade de instalações complexas de frameworks de terceiros (Lutz 2010);
- **NumPy:** Fundamental para a manipulação de arrays numéricos. No contexto da aplicação, o NumPy é utilizado para ler arquivos binários de saída do processamento (ex: dados da função custo) e estruturá-los para visualização (Oliphant 2015);
- **Matplotlib:** Utilizada para a plotagem de gráficos dentro da interface. A ferramenta gera visualizações da convergência do algoritmo (função custo log10 versus iterações), permitindo ao usuário monitorar a qualidade da inversão;
- **Subprocess:** Este módulo é essencial para a integração com o backend. Ele permite que a interface Python acione executáveis externos e comandos do sistema, como chamadas via MPI (Message Passing Interface) para processamento paralelo, capturando seus retornos e códigos de erro.

5.3 Gerenciamento de Concorrência (Threading)

Um desafio comum em interfaces gráficas que controlam processos científicos demorados é o "congelamento"da tela durante a execução. Como o algoritmo de estereoto-

mografia demanda alto custo computacional, sua execução na thread principal da GUI tornaria a interface irresponsiva.

Para solucionar este problema, implementou-se o uso da biblioteca threading. As chamadas para os processos de inversão e cálculos pesados são encapsuladas em threads separadas. Isso permite que a interface gráfica permaneça ativa e responsiva, possibilitando, por exemplo, que o usuário visualize resultados de convergência atualizados em tempo real.

5.4 Módulo de Picking Automático

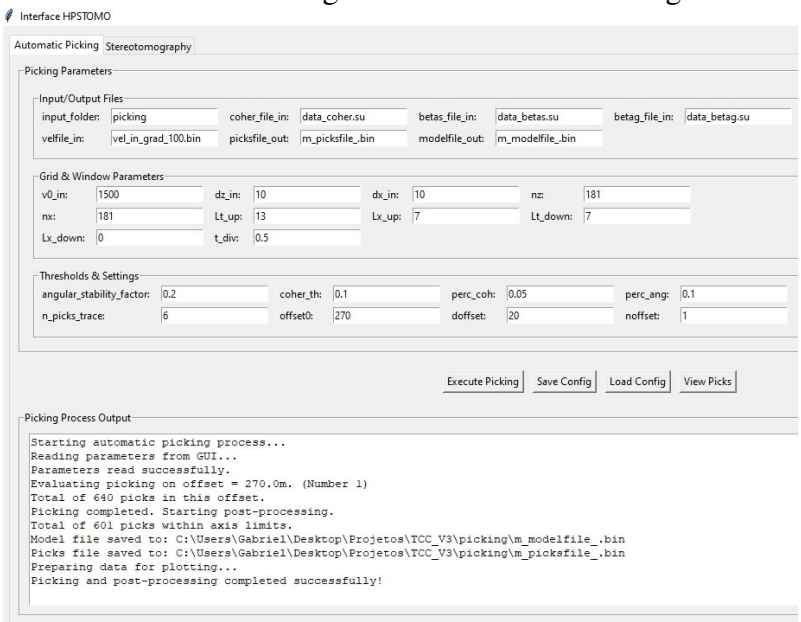
Uma das inovações centrais desta implementação é a aba dedicada ao Picking Automático. Esta funcionalidade foi desenvolvida para automatizar a extração dos parâmetros cinemáticos (t, p) necessários para a estereotomografia.O módulo de picking foi estruturado de forma a carregar scripts específicos localizados em subdiretórios da aplicação, garantindo modularidade. A lógica implementada utiliza funções matemáticas fundamentais (importadas via módulo math) para analisar a coerência dos sinais sísmicos.

Na interface, esta aba permite ao usuário:

- Carregar os dados sísmicos brutos;
- Visualizar os traços e aplicar algoritmos de detecção automática de eventos;
- Exportar os resultados no formato exigido pelo algoritmo de inversão tomográfica.

A integração deste módulo na mesma janela da inversão elimina a necessidade de alternância entre diferentes softwares, centralizando o fluxo de trabalho.

Figura 5.2: Tela da interface gráfica mostrando o Picking Automático.



Fonte: Elaborado pelo autor (2025)

A Figura 5.2 mostra a aba do sistema dedicada a execução do Picking Automático, onde é possível, selecionar e ajustar os parâmetros, executar, carregar e visualizar os gráficos da aplicação.

5.5 Módulo de Estereotomografia

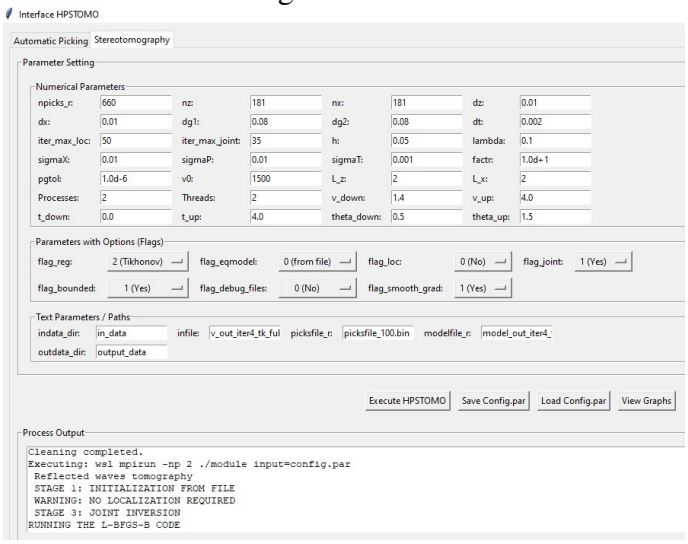
É o responsável pela orquestração da inversão estereotomográfica propriamente dita. Devido à alta carga computacional exigida pelo cálculo de traçamento de raios e pela otimização do modelo de velocidades, a implementação deste módulo seguiu a estratégia de desacoplamento entre a interface de controle (frontend) e o núcleo de cálculo (backend).

A implementação deste módulo utiliza a biblioteca Tkinter para criar uma camada de abstração sobre esses arquivos. Através de widgets de entrada (Entry, Combobox e Checkbutton), o usuário define visualmente os parâmetros da inversão, tais como:

- Número máximo de iterações e critérios de parada;
- Pesos de suavização para os componentes do modelo (velocidade e perturbações);
- Número de processadores a serem utilizados no processamento paralelo.

O script Python coleta esses valores, valida a integridade dos dados (ex: impedindo a entrada de caracteres não numéricos em campos de iteração) e constrói dinamicamente a chamada de execução. Essa abordagem segue o princípio de "prevenção de erros"(error prevention) (Nielsen 1993) para interfaces de usuário, garantindo que o backend receba apenas parâmetros válidos.

Figura 5.3: Tela da interface gráfica mostrando a Estereotomografia.



Fonte: Elaborado pelo autor (2025)

A Figura 5.3 mostra a aba do sistema dedicada a execução da estereotomografia, onde é possível, selecionar e ajustar os parâmetros, executar, carregar e visualizar os gráficos da aplicação.

5.6 Visualização de Resultados e Controle de Qualidade

A ferramenta implementa métodos específicos para a leitura e plotagem de arquivos de controle de qualidade (QC). O método de plotagem foi desenvolvido para ser robusto a falhas, incluindo tratamentos de exceção que verificam a existência de dados válidos (positivos e não nulos) antes de aplicar operações logarítmicas para a visualização da função custo.

Utilizando o matplotlib integrado ao Tkinter, o sistema gera gráficos de linha com marcadores que ilustram a evolução da otimização ao longo das iterações. Caso o arquivo de dados esteja vazio ou corrompido, situação comum em execuções falhas de processos externos, o sistema alerta o usuário através de caixas de mensagem (messagebox), garantindo que erros no backend sejam comunicados de forma clara no frontend.

Capítulo 6

Experimentos e Resultados

Este capítulo apresenta os testes realizados para validar a eficácia, a usabilidade e a robustez da Interface Gráfica (GUI) desenvolvida. Os experimentos foram conduzidos em duas etapas principais: a validação do módulo de pré-processamento (Picking Automático) e a validação do fluxo completo de inversão estereotomográfica. Para os testes, utilizou-se um conjunto de dados sísmicos sintéticos, permitindo um controle preciso sobre os parâmetros esperados.

6.1 Metodologia de Validação

A validação do software seguiu os critérios de teste de sistema para aplicações científicas (Kan 2003). O ambiente de teste consistiu em uma estação de trabalho equipada com sistema operacional Windows e Linux, com as bibliotecas Python (Tkinter, NumPy, Matplotlib) devidamente configuradas e o executável da estereotomografia compilado com suporte a MPI (Message Passing Interface) e OpenMP (Open Multi-Processing).

O objetivo dos experimentos foi verificar:

- Responsividade da interface durante operações pesadas;
- Capacidade do módulo de picking em identificar eventos coerentes;
- Correta comunicação entre a GUI e o backend numérico;
- Visualização adequada da convergência da função custo.

6.2 Validação do Módulo de Picking Automático

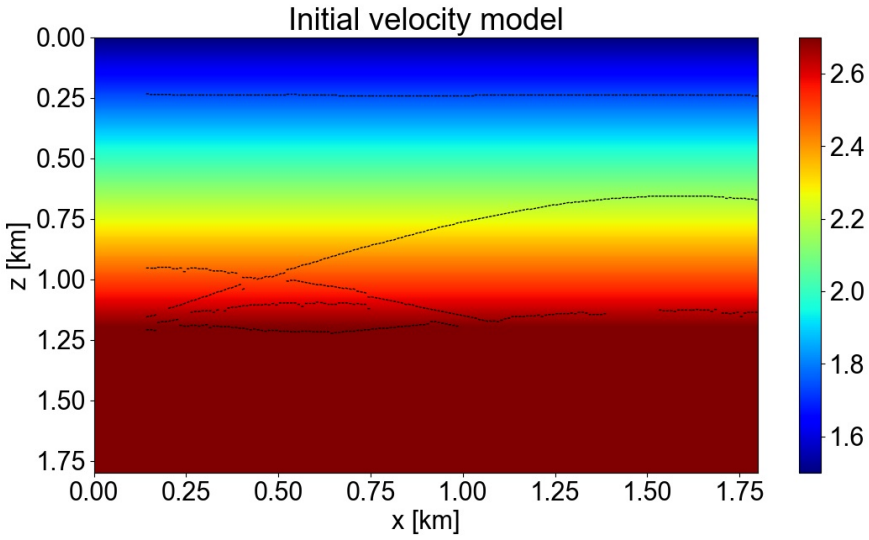
O primeiro experimento focou na aba de Picking Automático. Para esta validação, a entrada de dados consistiu em um conjunto de dados sísmicos sintéticos brutos, organizados em famílias de tiro comum (Common Shot Gathers). Este volume de dados é composto por matrizes numéricas que representam as amplitudes do campo de onda registrado, totalizando milhares de traços sísmicos que, se processados manualmente, demandariam semanas de trabalho de interpretação.

O objetivo desta etapa é reduzir essa massa de dados de amplitude para extrair apenas os atributos cinemáticos fundamentais: os pares de tempo de trânsito (t) e inclinação (p), essenciais para a construção do vetor de dados da estereotomografia (Billette & Lambaré

1998).Ao carregar o arquivo sísmico na interface, o algoritmo de detecção automática, baseado na coerência de fase e amplitude (Taner et al. 1979), foi executado sobre o volume de dados.Observou-se que a interface gráfica permitiu a visualização imediata dos traços sísmicos.

Após a execução do picking, o software gerou uma saída visual, sobrepondo os eventos identificados diretamente sobre a imagem sísmica, permitindo o controle de qualidade imediato pelo usuário. Dados sísmicos brutos foram transformados em um conjunto de parâmetros cinemáticos otimizado para o processamento tomográfico.

Figura 6.1: Modelo de Velocidade gerado pelo Picking Automático.



Fonte: Elaborado pelo autor (2025)

Conforme observado na figura 6.1 a ferramenta foi capaz de detectar as principais reflexões do modelo sintético com precisão aceitável. A funcionalidade de exportação gerou os arquivos de entrada formatados corretamente para o algoritmo de inversão, eliminando a necessidade de formatação manual e reduzindo a incidência de erros humanos, um ganho significativo de usabilidade.

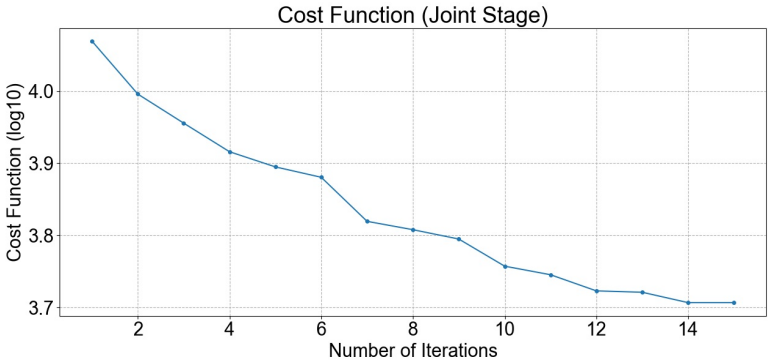
6.3 Execução da Inversão e Inspeção da Convergência

O segundo conjunto de experimentos visou testar o "coração"da ferramenta: a execução do algoritmo de estereotomografia via GUI. Utilizando os dados gerados na etapa de picking, configurou-se a inversão através dos painéis de parâmetros da interface. Foram definidos parâmetros de regularização e o número máximo de iterações.

O sistema foi configurado para executar em paralelo utilizando múltiplos núcleos de processamento via mpirun. Observou-se que ao iniciar o processamento, a interface permaneceu responsiva ("não travou"), validando a implementação da arquitetura de multithreading discutida no Capítulo 4. O usuário manteve a capacidade de navegar entre as abas e interagir com janelas enquanto o processo numérico ocorria em segundo plano.

Ao analisar a convergência (Controle de Qualidade), através da visualização da evolução da função custo (J), observou-se que o algoritmo de estereotomografia busca minimizar a diferença entre os dados observados e os dados calculados pelo modelo (Billette & Lambaré 1998). A GUI capturou periodicamente os dados de saída do backend e gerou, em tempo real, o gráfico de convergência ($Log_{10}(J)$ versus Iterações).

Figura 6.2: Modelo da Função Custo gerado pela inversão.

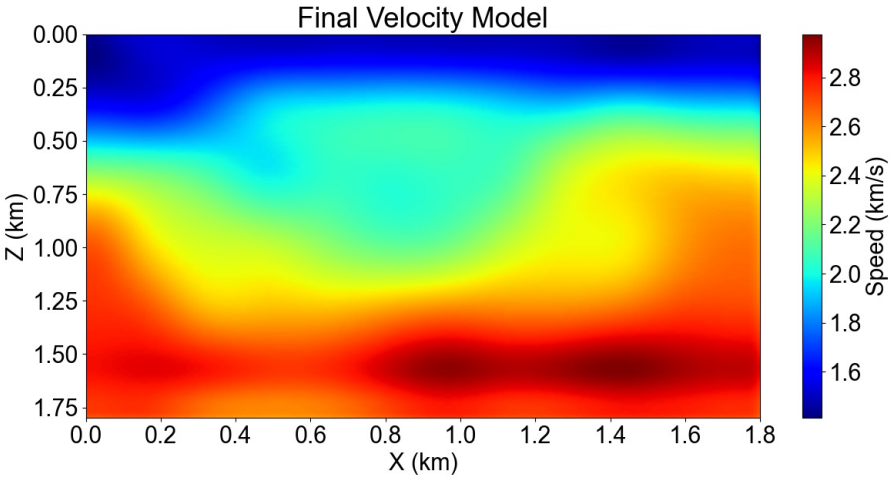


Fonte: Elaborado pelo autor (2025)

Conforme ilustrado na Figura 6.2, o gráfico exibiu uma curva descendente, indicando que o algoritmo estava convergindo adequadamente para uma solução estável. A implementação do tratamento de dados via NumPy garantiu que valores espúrios ou nulos não quebrassem a visualização, demonstrando a robustez do código de plotagem.

Além da análise numérica da função custo, o processo de inversão gera automaticamente os mapas de velocidade do meio, que servem como os principais parâmetros visuais de qualidade do imageamento. A ferramenta foi capaz de ler as matrizes resultantes e plotar o Modelo de Velocidade Final, conforme apresentado na Figura 6.3.

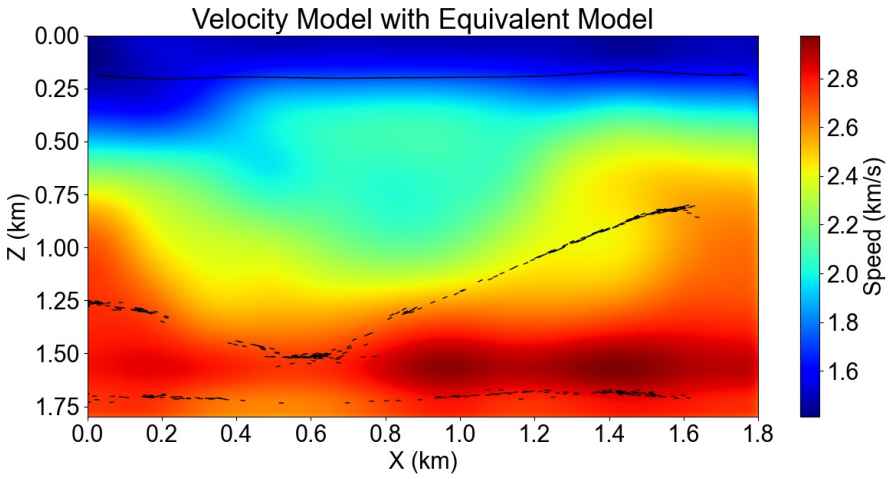
Figura 6.3: Modelo de Velocidade Final gerado pela inversão.



Fonte: Elaborado pelo autor (2025)

Simultaneamente, para fins de comparação e validação da suavidade do modelo, o software gerou o Modelo de Velocidade Equivalente, exibido na Figura 6.4. A inspeção visual destes dois resultados permite ao geofísico validar se as estruturas geológicas recuperadas são coerentes com o esperado para a região investigada.

Figura 6.4: Modelo de Velocidade Equivalente gerado pela inversão.



Fonte: Elaborado pelo autor (2025)

6.4 Análise de Usabilidade e Desempenho

A avaliação final considerou a experiência do usuário (UX) em comparação com o fluxo de trabalho tradicional via linha de comando (CLI). Observou-se que os seguintes critérios:

- **Centralização:** Todas as etapas (leitura, picking, configuração, execução e QC) foram realizadas em uma única janela, reduzindo o tempo de setup do projeto;
- **Feedback Visual:** A presença de logs de texto na interface e gráficos de convergência forneceu transparência ao processo, algo inexistente na execução cega via terminal;
- **Prevenção de Erros:** A interface bloqueou a inserção de parâmetros inválidos (ex: texto em campos numéricos) em diversos pontos, atuando como uma barreira de segurança antes da execução do código de alto desempenho;
- **Desempenho no Carregamento de Grandes Volumes de Dados:** A interface demonstrou capacidade estável e responsiva ao lidar com arquivos sísmicos extensos, permitindo a leitura, visualização e manipulação de grandes gathers sem travamentos. Esse desempenho reforça a adequação da ferramenta para cenários reais de processamento, onde o volume de dados pode chegar a dezenas ou centenas de megabytes.

Em suma, os resultados demonstram que a ferramenta desenvolvida não apenas funciona tecnicamente, integrando Python e Fortran/MPI com sucesso, mas também atinge o

objetivo de tornar a estereotomografia mais acessível e operacionalmente eficiente, sobretudo ao oferecer um ambiente capaz de lidar com grandes conjuntos de dados de forma fluida e organizada.

Capítulo 7

Conclusão

O presente trabalho alcançou seu objetivo principal ao desenvolver e implementar uma Interface Gráfica de Usuário (GUI) funcional para o algoritmo de estereotomografia, preenchendo a lacuna existente entre a alta performance computacional dos códigos de inversão e a necessidade de usabilidade e interatividade no fluxo de trabalho geofísico.

A utilização da linguagem Python, em conjunto com a biblioteca Tkinter, provou ser uma escolha estratégica assertiva. A arquitetura desenvolvida permitiu que a aplicação atuasse eficientemente como um gerenciador (wrapper) de processos, orquestrando a execução de códigos legados em Fortran via MPI sem comprometer a experiência do usuário. A implementação de técnicas de concorrência (multithreading) foi decisiva para garantir que a interface permanecesse responsiva durante o processamento intensivo, permitindo o monitoramento da convergência da função custo em tempo real — uma funcionalidade que supera as limitações das abordagens tradicionais baseadas em linha de comando.

Um dos resultados mais expressivos deste trabalho foi a integração do módulo de Picking Automático. A centralização desta etapa na mesma plataforma da inversão eliminou a necessidade de alternância entre múltiplos softwares e reduziu drasticamente o tempo de preparação dos dados. Os testes demonstraram que a automatização da seleção dos parâmetros cinemáticos (tempo e inclinação), aliada à capacidade de inspeção visual dos traços sísmicos, não apenas acelerou o fluxo de trabalho, mas também introduziu maior reprodutibilidade ao processo, minimizando a subjetividade inerente ao picking manual.

Conclui-se, portanto, que a ferramenta desenvolvida contribui significativamente para a democratização do uso da estereotomografia. Ao abstrair a complexidade da parametrização e do gerenciamento de arquivos, o software permite que geocientistas foquem na análise geológica e na qualidade do modelo final, transformando um código originalmente restrito à pesquisa em uma aplicação acessível ao ensino e à indústria.

Além das contribuições imediatas, este trabalho abre espaço para uma série de desenvolvimentos futuros. Entre eles, destaca-se a integração do método Common-Reflection-Surface (CRS) na interface, ampliando o escopo da aplicação e permitindo que, a partir de um dado sísmico bruto, seja possível realizar o fluxo completo de tomografia de forma independente. Outro avanço relevante envolve a adaptação da GUI para execução em ambientes de alto desempenho, possibilitando que a interface seja utilizada diretamente em supercomputadores, tanto para visualização quanto para gestão da execução. Por fim, a integração com sistemas de filas, como sbatch (SLURM), permitiria que o usuário enviasse e monitorasse tarefas distribuídas diretamente pela GUI, aproximando ainda mais a

ferramenta dos fluxos de trabalho profissionais utilizados em grandes centros de processamento sísmico.

Assim, o projeto não apenas entrega uma solução prática e funcional, mas estabelece uma base sólida para futuras extensões que poderão fortalecer sua aplicação no meio acadêmico e industrial.

Referências Bibliográficas

Barros, Tiago, Renato Lopes & Hervé Chauris (2022), 'Towards a more robust input for stereotomography', *Geophysical Prospecting* **70**(3), 502–524.
URL: *https://doi.org/10.1111/1365-2478.13181*

Bian, C., W. Geng, P. Yang, P. Sun, G. Xue & H. Lin (2022), An automatic velocity picking method based on object detection, *em* 'Second International Meeting for Applied Geoscience & Energy', pp. 3329–3333.
URL: *https://doi.org/10.1190/image2022-3745217.1*

Billette, F. J. & G. Lambaré (1998), 'Velocity macro-model estimation from seismic reflection data by stereotomography', *Geophysical Journal International* **135**(2), 671–680.
URL: *https://academic.oup.com/gji/article/135/2/671/781597*

Chauris, Herve & Mark Noble (2001), 'Two-dimensional velocity macro model estimation from seismic reflection data by local differential semblance optimization: Applications to synthetic and real data sets', *Geophysical Journal International* **144**(1), 14–26.
URL: *https://doi.org/10.1046/j.1365-246x.2001.00279.x*

Huang, Ya-Lin, Chia-Ying Hsieh, Jian-Xue Huang & Chun-Shu Wei (2022), 'Exbrainable: An open-source gui for cnn-based eeg decoding and model interpretation', *arXiv preprint arXiv:2201.04065* .
URL: *https://arxiv.org/abs/2201.04065*

Kan, Stephen H. (2003), *Metrics and Models in Software Quality Engineering*, 2ª edição, Addison-Wesley Professional, Boston, MA.
URL: *https://books.google.com.br/books/about/Metrics$_{a}nd_{M}odels_{i}n_{S}oftware_{Q}uality_{E}$.html?id = $EaefcL3pWJYC$*

Lutz, Mark (2010), *Programming Python: Powerful Object-Oriented Programming*, 4ª edição, O'Reilly Media.
URL: *https://books.google.com.br/books?id=ePyeNz2Eoy8Cprintsec=frontcoverredir$_{e}sc = yv = onepageqf = false$*

Morse, D. C. & N. T. Middleton (1994), A graphical user interface for geotomography, *em* 'Proceedings of 1994 IEEE Industry Applications Society Annual Meeting', pp. 2148–2155.
URL: *https://doi.org/10.1109/ias.1994.377727*

Nielsen, Jakob (1993), *Usability Engineering*, 1ª edição, Morgan Kaufmann, San Francisco, CA.
URL: *https://www.nngroup.com/books/usability-engineering/*

Oliphant, Travis E. (2015), *A Guide to NumPy*, 2ª edição, CreateSpace Independent Publishing Platform, USA.
URL: *https://www.google.com.br/books/edition/Guide$_t o_N umPy/g58ljgEACAAJ?hl = pt - BR$*

Richards, Tyler (2023), *Streamlit for Data Science: Create Interactive Data Apps in Python*, 2ª edição, Packt Publishing Limited.
URL: *https://www.angusrobertson.com.au/books/streamlit-for-data-science-tyler-richards/p/9781803248226*

Sabbione, Juan I. & Danilo R. Velis (2010), 'Automatic first-breaks picking: New strategies and algorithms', *Geophysics* **75**(4), V67–V76.
URL: *https://doi.org/10.1190/1.3463703*

Siahaan, Andysah Putera Utama, Afif Yasri, Ramlan Marbun, Muhammad Daffa Alfikri & Khairil Putra (2025), 'Design of gui-based digital image processing applications in python using opencv and tkinter', *World Journal of Advanced Engineering Technology and Sciences* **16**(1), 049–055.
URL: *https://doi.org/10.30574/wjaets.2025.16.1.1196*

Taner, M. T., F. Koehler & R. E. Sheriff (1979), 'Complex seismic trace analysis', *Geophysics* **44**(6), 1041–1063.
URL: *https://doi.org/10.1190/1.1440994*

Wilson, Greg, D. A. Aruliah, C. Titus Brown, Neil P. Chue Hong, Matt Davis, Richard T. Guy, Steven H. D. Haddock, Kathryn D. Huff, Ian M. Mitchell, Mark D. Plumbley, Ben Waugh, Ethan P. White & Paul Wilson (2014), 'Best practices for scientific computing', *PLOS Biology* **12**(1), e1001745.
URL: *https://doi.org/10.1371/journal.pbio.1001745*

Yilmaz,
"Ozdoğan (2001), *Seismic Data Analysis: Processing, Inversion, and Interpretation of Seismic Data*, Society of Exploration Geophysicists.
URL: *https://library.seg.org/doi/book/10.1190/1.9781560801580*

Zamorouev, Alexander (1999), Automatic picking of seismic events, *em* 'SEG Technical Program Expanded Abstracts', pp. 1158–1161.
URL: *https://doi.org/10.1190/1.1820708*

Zeyen, Hermann & Emmanuel Léger (2024), 'Pyrefra – refraction seismic data treatment and inversion', *Computers Geosciences* **185**, 105556.
URL: *http://dx.doi.org/10.1016/j.cageo.2024.105556*